



Frontend handover

The product app, the marketing site, and exactly how the two are wired together. Written to be read once and answered from.

Itqan • the signed in product app
React, TypeScript and Vite • Arabic first, RTL native, bilingual
Generated from **Onboarding/README.md**

Section 6 states plainly what is real and what is simulated. Read it before any demo.

React + TypeScript + Vite. Arabic first, RTL native, bilingual, responsive, light and dark.

This is the **signed in half** of Itqan. The marketing site (`itqan-website`) is the other half and owns the front door: the landing pages, and log in and sign up. This app owns everything after that: the onboarding flow and the dashboard.

Read this before a demo. §6 says exactly what is real and what is simulated. It is the section a judge's questions will land on, and being straight about it is the whole brand.

1. Running it

```
cd itqan-website && npm run build      # once, and after any site change
cd ../Onboarding && npm install && npm run dev -- --port 4333
```

Open **`http://localhost:4333/`** – that is the marketing site. Log in there and you land in the app. Opening `/app/` directly is fine too; with no session it sends you to the site's login page.

In development a Vite plugin (`dev/site-plugin.ts`) serves **both halves on one origin** and implements the API. That is why you build the site first: the plugin serves its built output.

Test accounts

Password for both: `itqan1234`

Email	What it exercises
<code>maryam@itqan.test</code>	Fresh account. Goes through the whole onboarding flow.
<code>nasser@itqan.test</code>	Already onboarded. Lands straight on the dashboard.

These live server side only and are never rendered into any page. Signing up through the site's own form also works and creates a real session.

Two handles for awkward paths

- Name a file `unreadable-x.pdf` to make the pipeline fail, so you can show the error state on purpose.
- Reload mid onboarding to see the "pick up where you left off" offer.

2. The journey, screen by screen

The order is the product argument, not just a flow.

1. **Upload** (`/upload`) – one drop area, not one slot per document type. Files arrive however they arrive and each lands as a row saying what Itqan thinks it is, correctable in one tap. The kind is guessed from the filename, because people name these files predictably. **The CV is the one required document;** everything else adds evidence. Hud is present here in his `analyzing` pose.
2. **Questions** (`/questions`) – asked *during* the pipeline wait, one at a time, so the wait does work instead of being dead time. Four questions: course pricing, work arrangement, preferred role, and whether to show adjacent roles. Every answer is optional; they only re-rank results, so refusing to answer never blocks anyone from their own transcript.
3. **Confirm** (`/confirm`) – the extracted details, editable. **Nothing downstream runs until the user confirms.** This is the first trust moment and the consent checkpoint. Anything the extraction was unsure about is labelled "Suggested – confirm" rather than stated as fact.
4. **Dashboard** (`/dashboard`) – where you stand, your highest yield skills, skills to unlock, then job postings. The readiness number never appears alone; it carries a sentence and a "how this is worked out" disclosure.
5. **Courses** (`/courses`) and **Job postings** (`/jobs`) – browsable, filterable, each item carrying its evidence and a real source link.

Routing guards, in `App.tsx`, encode two rules: onboarding is a gate a signed in user cannot skip by typing a URL, and it is a gate you pass **once** – someone who has finished can never be dropped back into it.

3. How the two halves connect

This app contains no login or signup screen, and never will. The site owns them. There is no `login()` or `signup()` in the API contract and no password anywhere in this codebase. A second sign in surface is a second thing to keep in step, and the first one to drift.

There are **three environments**, and the difference between them is real rather than incidental. This is the single most confusing thing in the project, so it is worth reading once properly.

	Who serves what	How the session travels
Dev	One origin. The Vite plugin serves the site at <code>/</code> and this app at <code>/app/</code> .	An ordinary cookie. Same origin, so it just works.
Vercel	Two separate projects on two domains.	A signed handoff token in the redirect URL. See below.
VPS (production)	One Caddy serves both: site at <code>/</code> , app at <code>/app/</code> .	An ordinary cookie again.

`src/lib/site.ts` is what knows the difference. `VITE_SITE_SAME_ORIGIN=1` (set by the deploy) makes links to the site relative; without it they are absolute, which is what Vercel needs.

The handoff token, and why it exists

On Vercel the site and the app are **separate deployments on separate domains**, and a cookie set on one domain cannot be read by the other. There is also no database, and serverless functions keep nothing between invocations, so the session cannot live on a server either.

So the user travels *inside a signed token*:

1. User submits the site's login form → `POST /api/placeholder/login`
2. Site validates, sets its own cookie → `200`
3. Site's form script redirects to `appUrl` → `/api/handoff` (same origin, so the cookie is sent)
4. `/api/handoff` reads the session, signs a short lived token, and redirects:
→ `https://app-.../?t=<token>`
5. The app boots, sees `?t=`, and sends it with its very first request
→ `GET /api/session?t=<token>`
6. That endpoint verifies the signature and sets a NORMAL session cookie on the app's own domain.
7. The client strips `?t=` from the address bar so a reload or a shared link cannot replay it.

The token is signed with HMAC SHA256 using a secret both projects share, lasts **two minutes**, and carries the user rather than a secret of its own. It cannot be forged without the secret, and once exchanged it is dead weight.

Steps 5 and 6 are one request on purpose: the app asks "who am I" and hands over the token together, so there is no window in which the app is loaded but not yet signed in.

The only change made to the website

`itqan-website/src/config.ts`, one value:

```
export const appUrl = '/api/handoff';
```

Nothing else in the site was touched – no markup, no styles, no copy, no components. That file's own comment describes it as "the single place to point the forms at the real API later", which is exactly what this is.

The site's existing form script already did:

```
const response = await fetch(form.action, { method: 'POST', body: new FormData(form) });
if (response.ok) window.location.assign(form.dataset.successUrl);
```

So a 200 plus a session cookie is the whole handshake. No new contract was invented.

What carries across

- **Session** – the cookie, however it got there.
- **Language** – the site's locale is in its URL; the login endpoint records which one was used and the app opens in it. Toggling language inside the app moves the same cookie, so the services answer in the new language too.
- **Theme** – the app reads and writes `itqan-theme`, the same key the site's toggle uses.
- **Assets** – the real `.webp` logo lockups and `.webm` mascot clips, straight from the site.

4. The API contract

`src/api/types.ts` is the seam between this UI and the agent services, and the only thing the screens depend on. `src/api/http.ts` is the client – one, always.

Endpoint	Does
<code>GET /api/session</code>	Who am I. Also exchanges a <code>?t=</code> handoff token.
<code>POST /api/logout</code>	Ends the session.
<code>GET PUT DELETE /api/onboarding/progress</code>	Resumable onboarding.
<code>POST /api/documents</code>	Upload one document, returns its record.
<code>POST /api/analysis</code>	Start the pipeline, returns a job id.
<code>GET /api/analysis/:jobId</code>	Poll it.
<code>POST /api/profile</code>	Confirm the profile. This is what <i>starts</i> matching.
<code>GET /api/dashboard</code> • <code>/api/jobs</code> • <code>/api/courses</code>	The results.

Two trust rules are enforced by the types, rather than by each screen remembering them:

- every extracted field travels with its own `confidence`, because four sequential agents compound error and a clean looking answer downstream would hide it;
- every recommendation carries `why` (the evidence chain) and a real `source` with a retrieval date, because nothing fabricated is displayable.

`TRUST_THRESHOLD` is `0.85`. Below it, output is labelled "Suggested – confirm" and never stated as fact.

The upload uses `XMLHttpRequest` rather than `fetch` for one reason: `fetch` still cannot report upload progress in any browser, and this is the one request where the user watches a bar.

5. Architecture

The agents are **API based**. This app never runs inference, never holds a model, and never does its own matching. It POSTs a document, polls a job, and GETs already ranked results. Swapping or re-hosting an agent changes nothing here as long as the shapes hold.

```
src/api/types.ts    the contract, and the only thing screens depend on
src/api/http.ts     the client
src/state/          auth, api, onboarding providers
src/screens/        the onboarding flow: Upload, Questions, Confirm, ResumeGate
src/app/            the signed in app: AppLayout, Dashboard, Jobs, Courses
src/components/     shared pieces, including Hud and the pipeline progress
src/i18n/           ar.json and en.json, kept at exact key parity
api/               serverless functions – the demo backend on Vercel
dev/site-plugin.ts  the dev backend: serves the site and implements the same endpoints
```

There used to be a mock client chosen when no API URL was set. It is gone: dev now serves real endpoints, so there is one code path in every environment. That branch had already caused a bug – the mock's `session()` always returned null, so a user who had just logged in on the site was bounced straight back to it.

6. What is real, and what is simulated

Read this before demoing. Being straight about it is the brand; a judge who catches an overclaim is the one user Itqan is built not to lose.

Real

- The whole front end: every screen, state, language, theme, and the responsive and RTL behaviour.
- Authentication as a mechanism. The token is genuinely HMAC signed, genuinely verified, genuinely expires, and the signature comparison is constant time.
- The session, the language handoff, the routing guards, and resumable onboarding.
- Upload progress. That bar is the real number of bytes sent.

Simulated, in this repo

- **There is no database.** Nothing persists anywhere. Everything that looks like storage is a cookie:
 - the session is a signed cookie;
 - "this account has finished onboarding" is not a row – `POST /api/profile` re-issues a **new session cookie with the flag flipped**;
 - onboarding progress is a cookie on the app's domain.
- **Uploaded bytes are discarded.** `POST /api/documents` keeps the name, size and kind, and throws the file away. There is no object store wired up, and pretending otherwise would hide that work rather than do it.
- **The pipeline is on a timer.** It runs for seven seconds and moves through reading → translating → matching → done. Progress is computed from a timestamp encoded in the job id, so it stays correct across cold starts and different serverless instances without any store.
- **The results are fixtures.** Real shaped on purpose: Omani employers, mixed Arabic and English strings, long names. And **deliberately imperfect** – the birth date comes back at 0.71 confidence and two skills are shaky, so "Suggested – confirm" is a normal path in the demo rather than a rare one.
- **Sign ups are not stored.** Anyone who signs up gets a valid session; their details ride in the signed token. There is nowhere to store them.

The important nuance

On the **VPS**, the deploy ships only the built static files. The `api/` folder is **not** deployed there – Caddy routes `/api` to the real backend, which is a separate Docker stack in another repo. So `Onboarding/api/*` is the demo backend **for Vercel**, and the contract in `src/api/types.ts` is what the real backend must implement.

7. Deployment

`.github/workflows/deploy.yml`, on push to `main`. It is gated on the full test suite because a single VPS has no per-PR preview URLs, so the tests are the safety net a CDN preview would otherwise be.

1. Typecheck, then **i18n parity** – Arabic and English must have exactly equal keys, because a missing key renders as a raw string like `dash.readinessUnknown` to a user.
2. Playwright E2E across Chromium, Firefox and WebKit.
3. Build both, with the deploy specific config:
 - `ITQAN_SITE_URL` bakes into canonical URLs, hreflang and the sitemap, so it must be present at build time;
 - `VITE_APP_BASE=/app/` mounts the app under `/app/` – both its asset URLs and its router basename. Wrong here means every asset 404s and the page is blank;
 - `VITE_SITE_SAME_ORIGIN=1` makes links to the site relative, because one host serves both.
4. Assemble one tree: site at the root, app under `/app/`.
5. Rsync to the VPS with `--delay-updates`, so no request is ever served a half copied site.
6. **Smoke test** `/`, `/app/` and `/api/health` – proving the deploy serves, rather than declaring success because rsync exited 0.

Set `ITQAN_AUTH_SECRET` on **both** projects to harden the token signing. The fallback keeps the demo working out of the box and is safe only because every account and record here is fake.

`api/_lib/auth.js` is **duplicated verbatim** in the site and the app. Two Vercel projects cannot import across their roots and a shared package would mean an install step. **If you change one, change the other.**

8. Design decisions, and why

Hud is absent from the confirmation screen, the dashboard, roles and courses. Locked brand rule: the mascot never appears beside a verdict, a score, a real match, or anything a user will act on, because a cartoon bird next to evidence reframes it as a guess. He belongs to the upload and reading screens, and his pose tracks the pipeline – `analyzing` while it runs, `error` if it fails, `celebrating` when it lands.

Instruction boxes are speech bubbles. A bubble reads as someone talking to you rather than a system notice. On wide screens he is large with the bubble beneath him, tail pointing up at his beak; on a phone it becomes a compact row with the tail pointing sideways, because vertical space is the scarce resource there. Every line he says also exists in the heading or body copy – he is `aria-hidden`, and a screen reader user must lose nothing.

One drop area, not one slot per document type. Six labelled dropzones would be a wall of empty boxes most users never fill. Only the CV is required, and the requirement is explained next to the button it blocks.

Dashboard order: where you stand → highest yield skills → skills to unlock → job postings. What is kept from the capability first argument is the framing rather than the sequence: the readiness number never appears alone, and gaps stay "to unlock" with a plus icon rather than a deficit in a danger hue.

The sidebar collapses to an icon rail (button, or ) , and the choice is remembered – someone who wants the wide view wants it every time. Labels are removed from the flow rather than hidden, so a screen reader cannot read a label nobody can see.

Nav order is the journey: Dashboard, Courses, Job postings. Postings sit last because they are the destination, not the starting point.

There is no hamburger. A drawer hiding three destinations costs a tap to save nothing. The account menu *is* a dropdown, because it holds a destructive action; it closes on Escape, on outside click, on selection, and when focus leaves.

Course prices are large and near the top. Cost is the second thing anyone wants to know, and a product that refuses affiliate commissions has no reason to bury it. Free is stated as a word, not "0.000 OMR", which reads as an oversight.

Both browse pages carry filters and a "look for new" action. These lists come from sources that change on their own schedule, so a user waiting on a posting needs a way to ask rather than reload and hope. The result is always reported, including "nothing new" – the honest and most common answer, and the one a silent spinner leaves ambiguous.

9. Interrupted onboarding

Progress saves after every meaningful change, debounced. Because it goes through the API rather than to this browser's local storage, starting on a phone and finishing on a laptop works.

Returning is **offered**, never forced. Landing on a later step with no in-memory state does not bounce the user to step one; it shows the offer on the step they were on, because a phone browser reloads a backgrounded tab and bouncing meant the screen "did not load" on exactly the devices that discard tabs.

10. Verified

Screenshots time out on this machine (the browser pane does not composite), so verification was done through DOM geometry and computed styles. That is stated rather than implied.

- Signed up and logged in **through the site's own forms**; both journeys walked end to end. Logout returns to the site's login page in the right language and clears the session.
- Endpoints checked directly: 401 on bad password, 409 on duplicate email, 401 with no session.
- **Contrast:** every text node on every route, light and dark, against composited backgrounds. Zero failures. **Targets:** every non-inline control $\geq 44\text{px}$.

- **RTL:** direction flips, layout mirrors, the bubble tail mirrors, Arabic gets its larger size and 1.75 line height, no physical CSS properties, mixed strings isolated with `<bdi>`.
 - No horizontal overflow at 375 / 585 / 1280. Both dictionaries at exact key parity, none unused.
-

11. Open items

- `--color-text-muted` **fails AA on two of three surfaces** — 4.32:1 on paper, 3.70:1 on sand, against a 4.5 floor; it passes only on white. Tokens are locked, so muted is used only inside white cards here. The design system needs a muted step that clears 4.5 on paper.
- `components.md` specifies gold text for ghost buttons, contradicting the locked "gold is never body text on light". The locked rule was followed; worth reconciling upstream.
- **The August 2026 palette revision has not been applied here.** `src/styles/tokens.css` still carries the retired gold `#D08C2F` and the old ramp. The final brand gold is `#F39F1C`.
- `react-router-dom@7.18.1` carries a high advisory for **RSC mode**, which is not reachable in library mode.
- Saved roles are component state; persisting them needs a real endpoint.
- Rubik loads from the Google Fonts CDN. Self host before production.
- In production the real API must implement the contract in `src/api/types.ts`. `dev/site-plugin.ts` does not ship, and `api/` is Vercel only.